

Engineering Reflection Questions

1. Why should source profiling occur before implementation of ingestion?
 - Profiling can reveal the shape, volume, and quality of the data that can allow engineering of certain features such as; designing accurate data contracts, choosing the correct ingestion strategies, and implement necessary validation checks.
2. What is the difference between source event time/updated_at and ingestion time?
 - Source event time shows the moment when the business event occurred or was modified in the source system. Ingestion time is the timestamp recorded when the data pipeline physically moves and writes that data into the destination system.
3. Why is event_id alone insufficient to decide which duplicate API record to keep in this exercise?
 - While `event_id` identifies that two records represent the same entity, it provides no chronological context. If a record was updated or duplicated in transit, you need an ordering field (like `updated_at` or extraction time) to deterministically decide which version is the most recent and should be kept.
4. Why must watermark state advance only after successful persistence?
 - If a pipeline updates its watermark state in the tracking file before the raw data is safely written to storage, a crash during writing can cause permanent data loss. The next run will start from the new watermark, skipping the records that failed to write.
5. What limitation does `updated_after > watermark` have when multiple source records can share exactly the same timestamp?
 - If multiple source records are generated at exactly timestamp = 100, and a pipeline pulls data but only captures a subset of them before saving 100 as the watermark, the next run querying `> 100` will permanently skip the remaining records that share that exact timestamp.
6. How is duplicate prevention related to idempotency?
 - Idempotency ensures that running a pipeline multiple times yields the same final state as running it once. Duplicate prevention (like relying on a primary key to upsert or overwrite records) is the mechanism that achieves this, ensuring that overlapping data pulls from re-runs do not multiply the row counts in the destination.
7. Why should the raw area preserve source values instead of applying business transformations?
 - The raw layer acts as an immutable, auditable historical record. If business rules change, or if a bug is discovered in your transformation logic, preserving the exact source data allows you to safely replay the historical data and rebuild downstream models without needing to re-extract from the source system (which may no longer have the old data).

8. How could querying a production OLTP source for profiling or extraction degrade the application?

- Operational databases are optimized for rapid, single-row transactions. Executing heavy analytical queries, large aggregations, or full-table scans consumes extensive CPU, memory, and disk input and output. This can lock critical tables and block live application traffic, leading to site slowdowns or outages for end users.

9. What would you change if the API had a rate limit of 60 requests per minute?

- I can implement a client-side timer (e.g., introducing a 1-second sleep between requests) to stay under the 60 requests/minute velocity threshold. Additionally, the pipeline needs error handling to catch HTTP 429 Too Many Requests responses, initiating backoff and retry logic instead of failing.

10. How would you extend this pipeline from a local raw area to PostgreSQL while preserving rerun safety?

- To preserve rerun safety when moving data from local files into PostgreSQL, you load the local file into a temporary table, then use an UPSERT operation. This ensures that re-running the pipeline updates existing rows instead of duplicating them.